

Eventus — an autonomous options agent

A multi-book options agent trading a live Alpaca paper account with no human in the loop. Featherless AI is the reasoning layer; Alpaca's MCP server is the tool surface; a deterministic risk engine is the only component in the system that can approve a trade.

Judged paper account PA3TSH9YTFJL · carry (resident) · intraday (transient) · earnings (own loop) · Qwen3-32B via Featherless

1 The AI logic — the model reasons, it never authorises

- **Discovery is Alpaca-native, no scraper.** At 09:15 the agent blends Alpaca news, screener movers and most-actives - rank-normalised within each source, then combined - into an attention snapshot of breadth, news velocity and volume rank, filtered for tradability into the day's candidate pool.
- **The macro call returns a regime, not a trade.** One Featherless call per session yields regime, volatility expectation, flagged symbols and a guidance verdict, plus a size multiplier bounded at 1.0, it can only ever **reduce** exposure. A deterministic breadth rule takes over on any error.
- **The agent drives Alpaca itself.** A tool loop lets the model query account, positions, orders, clock, last trade, option quote and news through Alpaca's MCP server and reason over what comes back. There is real autonomy, not a scripted pipeline with a model bolted on.
- **A critic scores and explains; it cannot approve.** Every candidate is scored against a 0.55 threshold with a required thesis, and the reasoning is written to the journal. No model sits anywhere in the exit path: position management is mechanical, routed back to the strategy that opened each position.
- **The earnings book proposes, a committed file confirms.** The model ranks names on implied move versus median reaction and calls direction (Qwen3-32B, confidence ≥ 0.55 , evidence cited; Qwen3-8B for triage), but expression follows the **sign of the implied-vs-realised divergence**, not the direction call: rich sells a condor, cheap buys a vertical, the middle band trades nothing.
- **Degradation is a design choice.** If the provider is unreachable the loop falls back to deterministic rules and keeps trading. Downtime costs P&L; a missing paragraph of reasoning does not.

2 The risk gates — a refusal is an artefact, not a dead end

- **Strategy gates, per book, run in parallel.** Carry requires rich premium ($iv_rank \geq 0.35$ AND $iv - rv \geq 0.03$ — either alone false-positives), a directionless tape ($adx \leq 25$), hard event exclusions, a macro check, and a cost gate capping the round trip at 20% of the credit. Intraday requires a VWAP cross inside one polling gap, a volume z-score in its time-of-day bucket, 3 of up to 7 confirmation votes, a one-sided RSI veto and a spread gate.
- **A temporal capital firewall is checked before anything else.** The resident carry book reserves its margin requirement first, up to 50% of equity; transient books lease only what Reg T leaves after that. Wrong book or wrong minute is a categorical refusal, no sizing is even performed.
- **One deterministic engine signs every ticket.** Plain Python, no discretion: session state and loss limits, defined-risk structure with a computable max loss and 1–4 legs, portfolio shape (position caps, duplicate-structure and re-entry cooldowns, per-book concentration), **sizing from max loss rather than from capital** so every position risks the same fraction, portfolio Greek caps where a block of exact zeros counts as missing rather than measured, and a cash buffer.
- **Liquidation is confirmed, not requested.** At the 15:15 cutoff working orders are cancelled first, then every transient position is closed and the book is polled until positions and resting orders are both zero. Failure halts trading. At 15:45 a carry verification re-reads fresh Reg T buying power — never a cached number — and a fail disables the transient books next session.
- **Every veto is written down.** Each rejection is journalled with the gate that fired, its reason and every metric it measured. That the intraday spread gate dominates is a finding about option microstructure, not a bug and it is evidence a judge can read.

3 Alpaca infrastructure and the Featherless API

- **Alpaca MCP server as the tool surface.** The agent speaks MCP to `alpaca-mcp-server` over stdio, with a synchronous facade over a background event loop so the rest of the codebase stays simple. The model's allowlist is seven read-only tools, kept small deliberately: schemas are re-sent every turn, and a wide surface costs latency for tools the agent never calls.
- **The read/write split is the safety model.** Reads come from MCP. Writes are first-party and stamped — the server's own order-placing tool is **never exposed to the model**, because an agent that can place a raw order can bypass the close-of-day verification the firewall exists to guarantee.
- **Execution over the Alpaca CLI.** Every order is a shell command that can be replayed, carries a deterministic client order id so a retry after an ambiguous failure is a check rather than a second fire, and is refused outright without a risk stamp. Multi-leg spreads go out atomically; the judged profile posts at the far touch with chasing off.
- **Alpaca market data, collapsed into one immutable snapshot per cycle.** 400 daily sessions for trend and realised vol, 1-minute bars for VWAP, option chains spanning the union of the books' expiry windows, and per-symbol news - fetched per symbol under a request-rate ceiling. Strategies read nothing else, so two books can never disagree about the tape.
- **Featherless is the live reasoning provider, not a decoration.** Every critic verdict, macro read and agent cycle in the judged account is produced there: `Qwen/Qwen3-32B` at temperature 0.2 with a fixed seed, so the same evidence yields the same verdict twice. It is written straight against the REST surface with `httpx` rather than an extra SDK; one fewer package to install on a competition host, and the wire format is small enough to own.
- **Built for serverless inference, not a lab.** Cold starts and rate limits are retried with exponential backoff; a non-retriable 4xx fails immediately rather than spending the cycle's latency budget on the same answer. Native JSON mode falls back to prompted JSON once per process. Its OpenAI-dialect tool loop is what lets Featherless drive the Alpaca MCP tools directly — **that is the bridge between the strategy layer and the broker**. One vendor and one key serve both the live loop and the backtest critic.

The agent decides. The deterministic layer still gates. Nothing reaches Alpaca without a signed stamp, and every refusal is recorded as evidence.

For a visualisation of the pipeline, please refer to the artefact here: <https://claude.ai/code/artifact/c308a2f1-bd4c-40ea-8995-0ecbfbbfbc68>